

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №23

Тема: Алгоритм Евкліда. Табулювання функції

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати алгоритм Евкліда для знаходження найбільшого спільного дільника (у варіантах із відніманням та залишками), розширений алгоритм Евкліда; навчитися табулювати функцію на відрізку з заданим кроком, враховувати область визначення, визначати екстремуми та відрізки зміни знака функції.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal), бібліотека matplotlib (для графіків).

Короткі теоретичні відомості

1. Найбільший спільний дільник (НСД) двох натуральних чисел a і b — найбільше число, на яке діляться обидва числа без остачі. Числа з $\text{НСД} = 1$ називають взаємно простими. Пов'язаний із ним **найменше спільне кратне**: $\text{НСК}(a, b) = a \cdot b / \text{НСД}(a, b)$.

2. Алгоритм Евкліда ґрунтується на властивості: $\text{НСД}(a, b) = \text{НСД}(b, a \bmod b)$, а $\text{НСД}(a, 0) = a$. Є два варіанти:

- **із відніманням:** поки $a \neq b$, від більшого числа віднімають менше; коли числа стануть рівними, вони дорівнюють НСД. Простий, але повільний, якщо числа дуже різні за величиною;
- **із залишками (ділення):** поки $b \neq 0$, виконують $(a, b) := (b, a \bmod b)$; відповідь — a . Значно швидший: кількість кроків не перевищує $\approx 5 \cdot (\text{кількість десяткових цифр меншого числа})$.

3. Складність. Варіант із залишками виконує $O(\log \min(a, b))$ кроків. За теоремою Ламе найгірший випадок досягається на сусідніх числах Фібоначчі $(F(n+1), F(n))$: кожне ділення дає частку 1, а залишок зменшується найповільніше. Варіант із відніманням у найгіршому випадку (наприклад, $(n, 1)$) виконує $n-1$ кроків.

4. Розширений алгоритм Евкліда знаходить разом з $d = \text{НСД}(a, b)$ такі цілі коефіцієнти x, y , що $a \cdot x + b \cdot y = d$ (тотожність Безу). Вони потрібні для розв'язання діофантових рівнянь і для знаходження оберненого елемента за модулем (криптографія). Рекурсивно: якщо $b = 0$, то $(d, x, y) = (a, 1, 0)$; інакше для $(d, x_1, y_1) = \text{ext}(b, a \bmod b)$ маємо $x = y_1, y = x_1 - (a // b) \cdot y_1$.

5. Табулювання функції – обчислення значень функції $y = f(x)$ у точках $x_0 = a$, $x_1 = a + h$, ..., $x_n = b$ відрізка $[a, b]$ із кроком h . Кількість точок: $n + 1$, де $n = (b - a) / h$. Результат оформлюють у вигляді таблиці « $x - f(x)$ ». Таблиці застосовують для побудови графіків, пошуку найбільшого/найменшого значень і для **відокремлення коренів**: якщо в сусідніх вузлах $f(x_i) \cdot f(x_{i+1}) < 0$, то між ними є принаймні один корінь (за умови неперервності функції).

6. Особливості реалізації. (а) Щоб уникнути накопичення похибки, значення x обчислюють за формулою $x_i = a + i \cdot h$ (з цілим лічильником i), а не багаторазовим додаванням $x += h$; (б) кількість кроків визначають округленням $\text{round}((b - a) / h)$; (в) перед обчисленням перевіряють **область визначення** (ділення на нуль, корінь із від'ємного числа, логарифм невід'ємного аргумента), а в таблиці такі точки позначають «не визначено».

Хід роботи

Завдання 1. Ручне трасування алгоритму Евкліда

Виконати вручну алгоритм Евкліда в обох варіантах та розширений алгоритм Евкліда.

Порядок виконання:

1. Знайдіть НСД(1071, 462) варіантом із залишками: побудуйте таблицю «крок — a — b — $a \bmod b$ » і запишіть відповідь (число, на якому зупиниться алгоритм).
2. Знайдіть НСД(48, 18) варіантом із відніманням: запишіть на кожному кроці пару (a, b) та кількість виконаних віднімань.
3. Обчисліть НСД(17, 5) та НСД(100, 75) обома способами й порівняйте кількість кроків.
4. Знайдіть НСК(48, 18) через НСД.
5. Виконайте розширений алгоритм Евкліда для (240, 46): запишіть послідовність ділень із частками, зробіть зворотну підстановку й знайдіть x, y такі, що $240x + 46y = \text{НСД}$; перевірте результат підстановкою.

Завдання 2. Ручне табулювання функції

Виконати табулювання двох функцій: $f(x) = x^2 - 3x + 2$ на відрізку $[0; 4]$ із кроком $h = 0,5$ та $g(x) = 1 / (x^2 - 4)$ на відрізку $[-3; 3]$ із кроком $h = 1$.

Порядок виконання:

1. Обчисліть кількість точок табулювання для кожної функції за формулою $n + 1 = (b - a) / h + 1$.
2. Складіть таблицю значень $f(x)$ у всіх вузлах (9 точок); результат подайте з точністю до 0,01.
3. За таблицею визначте найменше значення функції f і точку, де воно досягається; визначте вузли, у яких $f(x) = 0$.
4. Знайдіть відрізки зміни знака функції f (пари сусідніх вузлів, де $f(x_i) \cdot f(x_{i+1}) < 0$) і поясніть, чому в цьому прикладі жодного такого відрізка немає (корені збіглися з вузлами).
5. Складіть таблицю значень $g(x)$; для точок $x = \pm 2$ запишіть «не визначено» та поясніть причину.
6. Поясніть, чому для табулювання потрібно спочатку перевіряти область визначення.

Завдання 3. Програмна реалізація алгоритму Евкліда

Реалізувати мовою Python обидва варіанти алгоритму Евкліда з підрахунком кроків, НСК, розширений алгоритм та порівняти їх ефективність.

Порядок виконання:

1. Напишіть функцію `gcd_subtract(a, b)`, що виконує алгоритм із відніманням і повертає пару (НСД, кількість кроків).
2. Напишіть функцію `gcd_mod(a, b)` — варіант із залишками; також повертає пару (НСД, кількість кроків).
3. Напишіть функцію `lcm(a, b)` через НСК (використайте цілочисельне ділення).
4. Напишіть рекурсивну функцію `ext_gcd(a, b)`, що повертає трійку (d, x, y) з $a \cdot x + b \cdot y = d$.
5. Перевірте результати на числах із ручних розрахунків та порівняйте з функцією `math.gcd`; перевірте тотожність Безу для кількох пар.
6. Порівняйте кількість кроків обох варіантів для пар (1071, 462), (1 000 000, 7) та двох сусідніх чисел Фібоначчі (832 040, 514 229).

Приклад реалізації мовою Python:

python

```
import math

def gcd_subtract(a, b):
    steps = 0
    while a != b:
        if a > b:
            a -= b
        else:
            b -= a
        steps += 1
    return a, steps

def gcd_mod(a, b):
    steps = 0
    while b != 0:
        a, b = b, a % b
        steps += 1
    return a, steps

def lcm(a, b):
    return a // gcd_mod(a, b)[0] * b

def ext_gcd(a, b):
    if b == 0:
        return a, 1, 0
    d, x1, y1 = ext_gcd(b, a % b)
    return d, y1, x1 - (a // b) * y1

print("gcd_mod(1071, 462) =", gcd_mod(1071, 462))
print("gcd_subtract(48, 18) =", gcd_subtract(48, 18))
print("lcm(48, 18) =", lcm(48, 18))
d, x, y = ext_gcd(240, 46)
print(f"ext_gcd(240, 46): d = {d}, x = {x}, y = {y}; перевірка 240*({x}) + 46*({y}) = {240 * x + 46 * y}")

ok = all(gcd_mod(a, b)[0] == gcd_subtract(a, b)[0] == math.gcd(a, b)
         for a in range(1, 60) for b in range(1, 60))
print("Збіг з math.gcd для всіх пар 1..59:", ok)
```

```
print("\n      a      b      НСД  кроків(віднім.)  кроків(залишки)")
for a, b in ((1071, 462), (1000000, 7), (832040, 514229)):
    g1, s1 = gcd_subtract(a, b)
    g2, s2 = gcd_mod(a, b)
    print(f"{a:>8} {b:>10} {g1:>6} {s1:>16} {s2:>16}")
```

Результат виконання програми:

```
gcd_mod(1071, 462) = (21, 3)
gcd_subtract(48, 18) = (6, 4)
lcm(48, 18) = 144
ext_gcd(240, 46): d = 2, x = -9, y = 47; перевірка 240*(-9) + 46*(47) = 2
Збіг з math.gcd для всіх пар 1..59: True
```

a	b	НСД	кроків(віднім.)	кроків(залишки)
1071	462	21	11	3
1000000	7	1	142863	2
832040	514229	1	28	28

Висновок за результатами: результати збігаються з ручними обчисленнями: $\text{НСД}(1071, 462) = 21$ за три ділення, $\text{НСК}(48, 18) = 144$, а для розширеного алгоритму $240 \cdot (-9) + 46 \cdot 47 = 2$. Обидва варіанти дають однакові НСД для всіх перевірених пар, але для $(1\ 000\ 000, 7)$ варіант із відніманням виконує 142863 кроків, а варіант із залишками — лише 2. Навіть на сусідніх числах Фібоначчі, що є найгіршим випадком для алгоритму з залишками, потрібно лише 28 кроків, тобто кількість кроків справді зростає логарифмічно.

Завдання 4. Програмне табулювання функції та аналіз таблиці

Реалізувати універсальну функцію табулювання, дослідити накопичення похибки при багаторазовому додаванні кроку та проаналізувати таблицю функції $f(x) = x^3 - 3x + 1$ на відрізку $[-2; 2]$.

Порядок виконання:

1. Напишіть функцію `tabulate(f, a, b, h)`, що повертає списки вузлів `xs` та значень `ys`, обчислюючи $x_i = a + i \cdot h$ для $i = 0 \dots n$, де $n = \text{round}((b - a) / h)$; точки поза областю визначення позначають `None`.
2. Порівняйте два способи побудови вузлів для відрізків $[0; 1]$ та $[0; 0,3]$ із кроком $h = 0,1$: накопиченням `x += h` у циклі `while x <= b` та за формулою $a + i \cdot h$. Підрахуйте кількість точок і значення останнього вузла.
3. Виведіть таблицю для $f(x) = x^3 - 3x + 1$ на $[-2; 2]$ із кроком 0,25 у форматі з чотирма знаками після коми.
4. За таблицею знайдіть мінімальне та максимальне значення функції і відповідні точки, а також усі відрізки зміни знака (кандидати для пошуку коренів).
5. Побудуйте графік функції за допомогою `matplotlib` і позначте знайдені вузли; позначте відрізки зміни знака.
6. Виведіть таблицю для $g(x) = 1/(x^2 - 4)$ на $[-3; 3]$ із кроком 1 і переконайтеся, що в точках $x = \pm 2$ виводиться «не визначено».

Приклад реалізації мовою Python:

```
python
import matplotlib.pyplot as plt
```

```

def tabulate(f, a, b, h):
    n = round((b - a) / h)
    xs, ys = [], []
    for i in range(n + 1):
        x = a + i * h
        try:
            y = f(x)
        except ZeroDivisionError:
            y = None
        xs.append(x)
        ys.append(y)
    return xs, ys

def by_accumulation(a, b, h):
    pts, x = [], a
    while x <= b:
        pts.append(x)
        x += h
    return pts

for b in (1, 0.3):
    acc = by_accumulation(0, b, 0.1)
    xs, _ = tabulate(lambda t: t, 0, b, 0.1)
    print(f"[0; {b}], h = 0.1: накопичення -> {len(acc)} точок, останнє {acc[-1]!r}")
    print(f"'':14}формула a + i*h -> {len(xs)} точок, останнє {xs[-1]!r}")

f = lambda x: x ** 3 - 3 * x + 1
xs, ys = tabulate(f, -2, 2, 0.25)
print("\n      x      f(x)")
for x, y in zip(xs, ys):
    print(f"{x:>6.2f} {y:>9.4f}")

k_min = min(range(len(ys)), key=lambda i: ys[i])
k_max = max(range(len(ys)), key=lambda i: ys[i])
print(f"\nmin f = {ys[k_min]:.4f} при x = {xs[k_min]}; max f = {ys[k_max]:.4f} при x = {xs[k_max]}")
brackets = [(xs[i], xs[i + 1]) for i in range(len(xs) - 1) if ys[i] * ys[i + 1] < 0]
print("Відрізки зміни знака:", brackets)

g = lambda x: 1 / (x * x - 4)
for x, y in zip(*tabulate(g, -3, 3, 1)):
    print(f"g({x:>2}) = " + ("не визначено" if y is None else f"{y:.4f}"))

plt.figure(figsize=(7, 4))
plt.plot(xs, ys, "o-", label="f(x) = x³ - 3x + 1")
plt.axhline(0, color="gray", lw=0.8)
for lo, hi in brackets:
    plt.axvspan(lo, hi, color="orange", alpha=0.3)
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Табулювання функції та відрізки зміни знака")
plt.grid(alpha=0.4)
plt.legend()
plt.show()

```

Результат виконання програми:

```

[0; 1], h = 0.1: накопичення -> 11 точок, останнє 0.9999999999999999
      формула a + i*h -> 11 точок, останнє 1.0
[0; 0.3], h = 0.1: накопичення -> 3 точок, останнє 0.2
      формула a + i*h -> 4 точок, останнє 0.30000000000000004

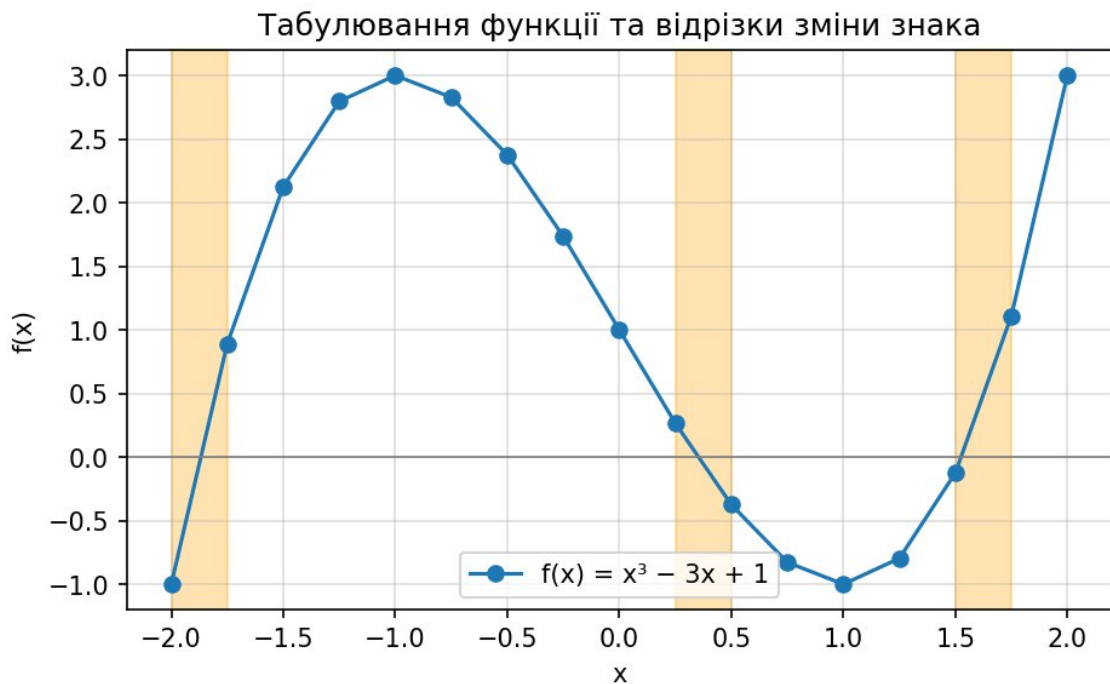
      x      f(x)
-2.00    -1.0000
-1.75     0.8906
-1.50     2.1250

```

-1.25	2.7969
-1.00	3.0000
-0.75	2.8281
-0.50	2.3750
-0.25	1.7344
0.00	1.0000
0.25	0.2656
0.50	-0.3750
0.75	-0.8281
1.00	-1.0000
1.25	-0.7969
1.50	-0.1250
1.75	1.1094
2.00	3.0000

$\min f = -1.0000$ при $x = -2.0$; $\max f = 3.0000$ при $x = -1.0$
 Відрізки зміни знака: $[(-2.0, -1.75), (0.25, 0.5), (1.5, 1.75)]$
 $g(-3) = 0.2000$
 $g(-2) = \text{не визначено}$
 $g(-1) = -0.3333$
 $g(0) = -0.2500$
 $g(1) = -0.3333$
 $g(2) = \text{не визначено}$
 $g(3) = 0.2000$

Графік порівняння:



Висновок за результатами: накопичення $x \pm 0.1$ призводить до помилок: на відріжку $[0; 1]$ останній вузол дорівнює 0.9999999999999999 замість 1 , а на відріжку $[0; 0.3]$ сума $0.1 + 0.1 + 0.1$ трохи перевищує 0.3 , тому остання точка **губиться** (3 точки замість 4). Формула $a + i \cdot h$ завжди дає правильну кількість вузлів, а її похибка не накопичується. Таблиця функції $x^3 - 3x + 1$ виявила 3 відрізки зміни знака, які містять три корені рівняння (приблизно -1.88 ; 0.35 та 1.53), а для функції $1/(x^2 - 4)$ у точках $x = \pm 2$ коректно виводиться «не визначено» замість аварійного завершення програми. Такі відрізки слугують початковими наближеннями для методів розв'язування рівнянь (практична робота №26).

Контрольні питання

1. Що таке НСД? Як пов'язані НСД і НСК двох чисел?
2. Сформулюйте ідею алгоритму Евкліда. На якій властивості він ґрунтується?
3. У чому різниця між варіантами алгоритму Евкліда з відніманням і з залишками? Який швидший і чому?
4. Яка складність алгоритму Евкліда з залишками? Який випадок є для нього найгіршим?
5. Що стверджує тотожність Безу і як її знаходить розширений алгоритм Евкліда? Де це застосовується?
6. Що таке табулювання функції? Як обчислити кількість точок таблиці?
7. Чому вузли табулювання краще обчислювати за формулою $a + i \cdot h$, а не накопиченням $x += h$?
8. Як за таблицею значень відокремити корені рівняння? Чому потрібно враховувати область визначення функції?

Зміст звіту

1. Тема, мета роботи.
2. Таблиці трасування алгоритму Евкліда та таблиці табулювання (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік функції з відрізками зміни знака.
5. Висновки.